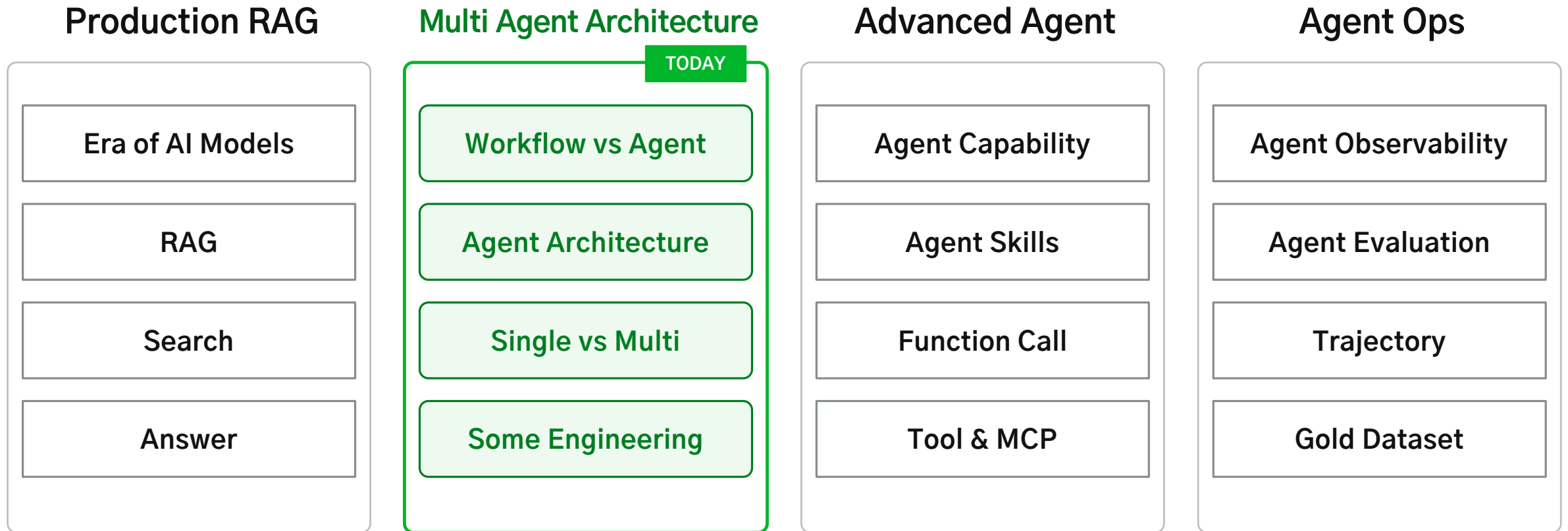

Multi Agent Architecture

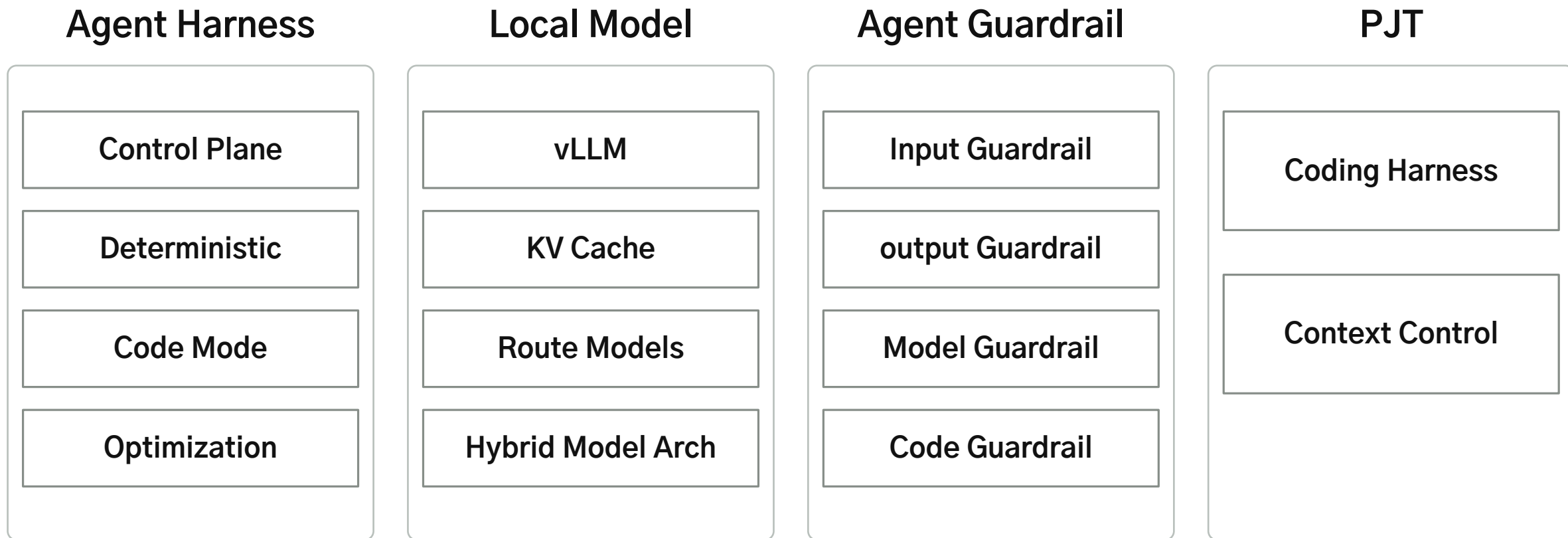
전현준

OT



오늘의 질문: “몇 개를 만들까?”보다 “어디에 판단을 맡길까?”

OT



오늘은 구조·경계·선택 근거, 이후에는 구현 심화·서빙·운영을 다룹니다.

목차

1. Workflow vs Agent

제어

2. AI Agent Architecture

실행 구조

3. Single Agent? Multi Agent?

분리 기준

4. 무슨무슨 Engineering

설계 범위

오늘 수업의 전체 방향

업무를 먼저 나누고, 필요한 곳에만 Agent를 배치합니다.



최종 결과물은 “에이전트 조직도”가 아니라, 이유를 설명할 수 있는 실행 구조입니다.

패턴 이름을 외우는 대신, 왜 이 구조를 선택했는지 설명합니다.

Chapter 1.

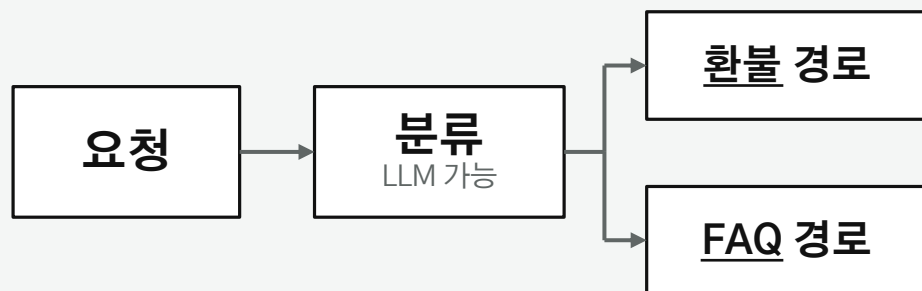
Workflow vs Agent

Workflow와 Agent의 차이는 “다음 행동의 결정권”

Workflow

CODE

코드가 정한 경로 안에서 실행합니다.

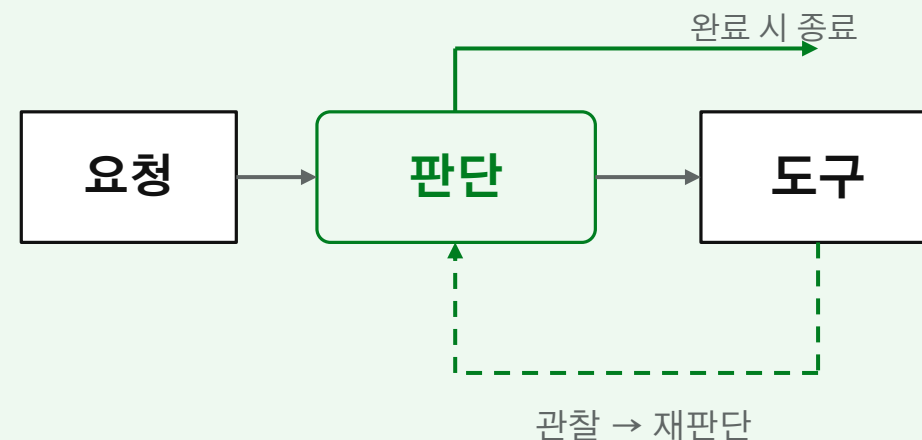


분기는 있어도 선택지는 코드로 정의

Agent

MODEL

모델이 결과를 보고 다음 행동을 선택합니다.

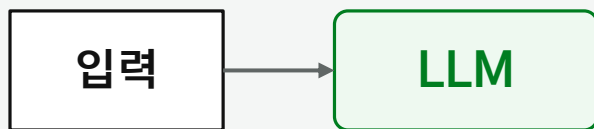


LLM을 사용한다고, 모두 Agent는 아닙니다.

LLM 호출, 도구 사용, Agent Loop를 구분해서 봅니다

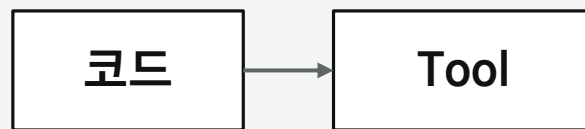
세 구조는 세대교체가 아니라, 업무에 따라 선택하는 실행 방식입니다.

01 LLM 호출



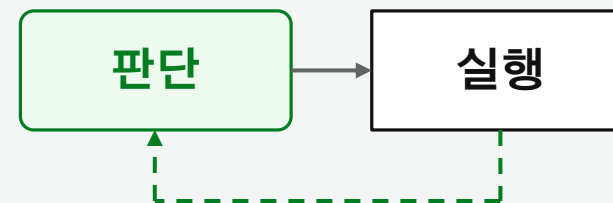
한 번의 생성 결과를 받습니다.

02 도구 사용



외부 기능 호출만으로
자율 실행구조 X

03 Agent Loop



관찰 후 계속할지
끝낼지 선택합니다.

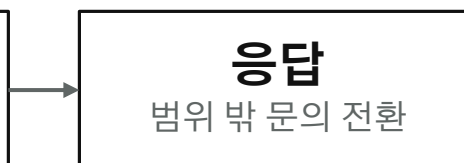
도구 호출 유무보다, 관찰 뒤의 행동 선택과 종료 조건을 확인합니다.

같은 고객 요청도 서로 다른 구조로 처리할 수 있습니다

가상 사례

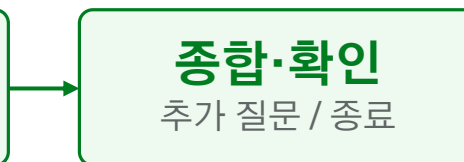
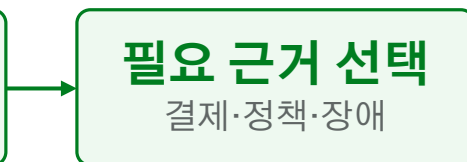
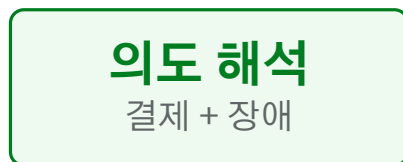
“결제가 두 번 됐어요. 취소하고 보상안도 함께 확인해 주세요.”

고정 경로



예외를 미리
정의해야 함

동적 조사

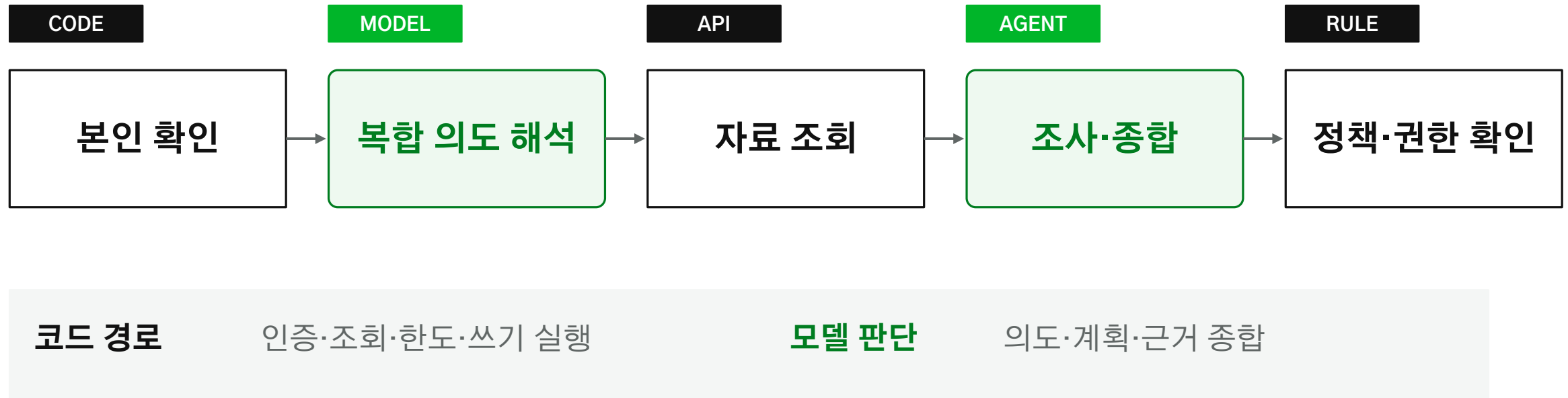


근거에 따라
조사 경로 변경

같은 업무를 놓고 구조를 바꾸면, 선택 이유와 비용을 비교할 수 있습니다.

Workflow 안에 Agent를 넣을 수 있습니다

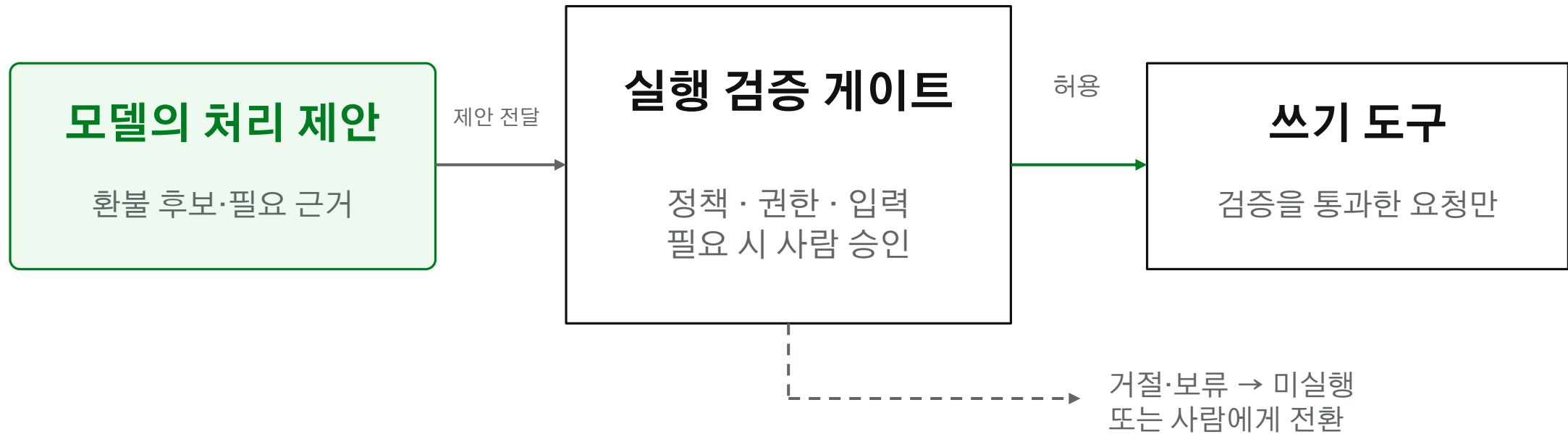
예측 가능한 절차는 고정하고, 모호한 판단만 모델에게 맡깁니다.



좋은 Agentic System은 “모든 노드가 Agent”인 시스템이 아닙니다.

모델의 제안과 시스템의 실행 권한은 다릅니다

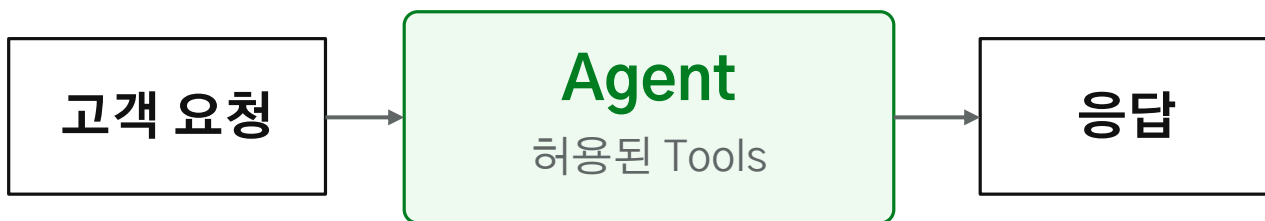
“가능할 것 같다”는 모델 출력이 곧 실행 허가는 아닙니다.



모델은 행동을 제안하고, 시스템은 허용된 행동만 실행합니다.

먼저 Single Agent 기준선을 만듭니다

작은 사례를 실행하고 성공 여부·시간·호출 수를 기록합니다.



관찰 질문

어떤 요청에서 실패했습니까?
정보 부족과 실행 오류를 구분했습니까?

측정 기록

task_success	성공 / 실패
total_latency	소요 시간
model / tool calls	각 호출 수
input / output tokens	토큰 수

실행 기록 + 실패 사례 1개 => 기준선 없이는 개선 여부도 알 수 없습니다.

Chapter 2.

AI Agent Architecture

패턴X, 먼저 업무의 상태와 의존관계를 그림니다

목표 → 필요한 상태 → 작업 → 선행 조건 → 종료 조건

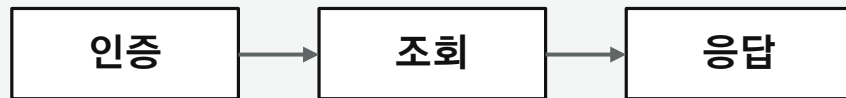


“Billing Agent를 만들자”보다 “무엇이 먼저 필요할까?”를 질문합니다.

복잡한 구조도 네 가지 실행 방식으로 읽을 수 있습니다

순차 Sequence

선행 결과가 있어야 다음 단계를 실행



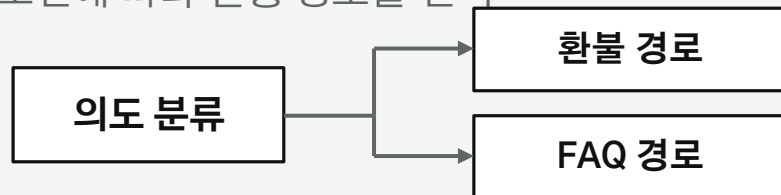
병렬 Parallel

독립적인 작업을 실행한 뒤 합류



분기 Branch

조건에 따라 실행 경로를 선택



루프 Loop

검증 후 재시도하거나 종료



새로운 패턴을 만나도, 순서·독립성·조건·종료부터 확인합니다.

패턴 이름을 실행 구조로 번역해 봅시다

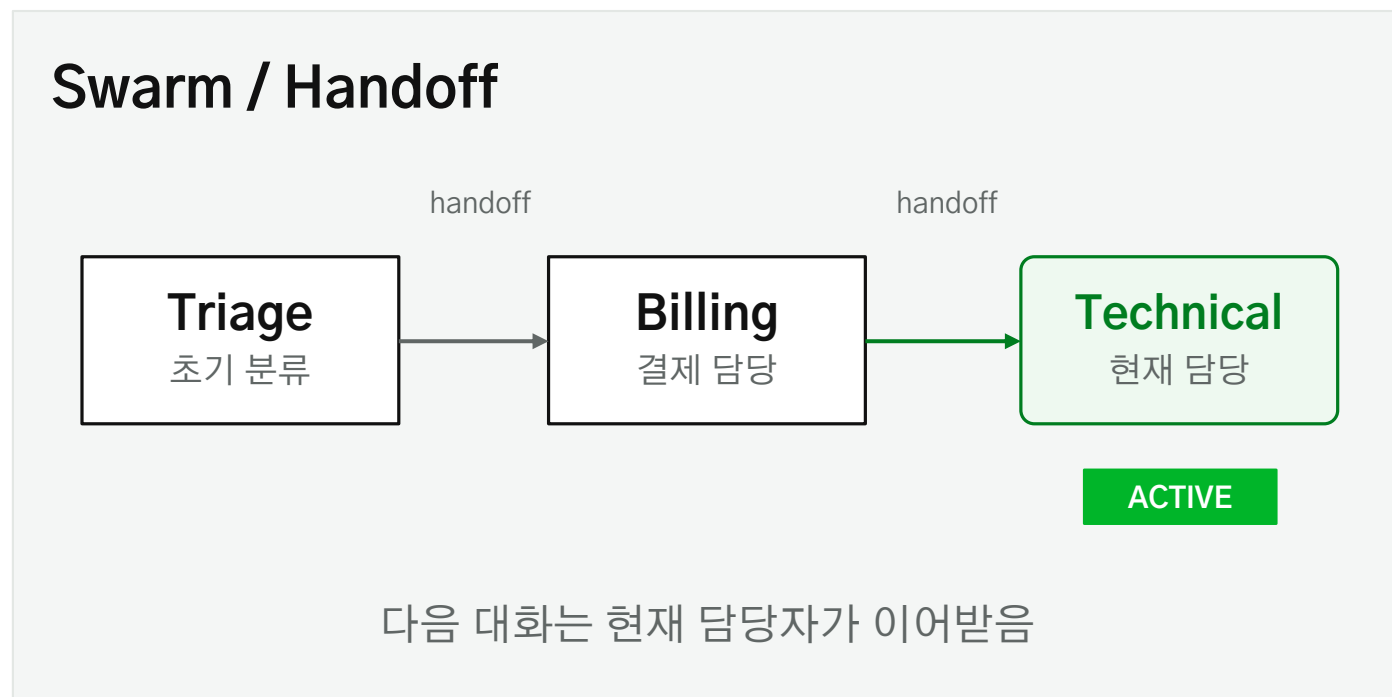
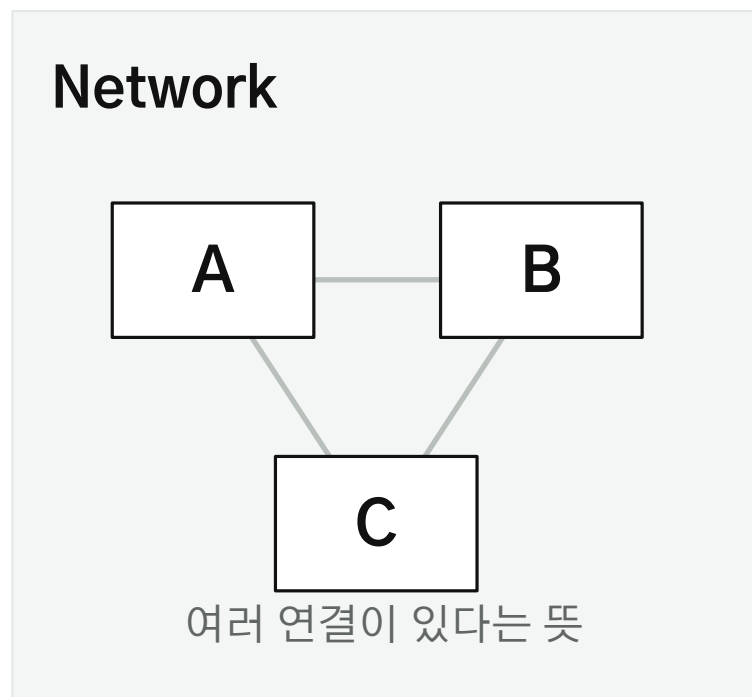
Supervisor와 Swarm 등 패턴들도 새로운 제어 구조가 전혀 아닙니다.

패턴	실행 구조로 읽기	확인할 질문
Router	조건 분기	어떤 경로로 보낼까?
Supervisor	분기 + 루프, 필요 시 병렬	누가 결과를 받고 다음 일을 정할까?
Orchestrator-Worker	분해 → 작업 실행 → 집계	어떻게 나누고 다시 합칠까?
Swarm / Handoff	담당자 전환 + 반복	누가 대화를 이어받을까?
Hierarchical Supervisor	위 구조를 계층으로 중첩	새 경계가 정말 필요한가?

Plan-and-Execute는 계획·실행·재계획을 바라보는 별도의 관점입니다.

Network와 Swarm | 연결 형태와 제어권 전환을 구분합니다

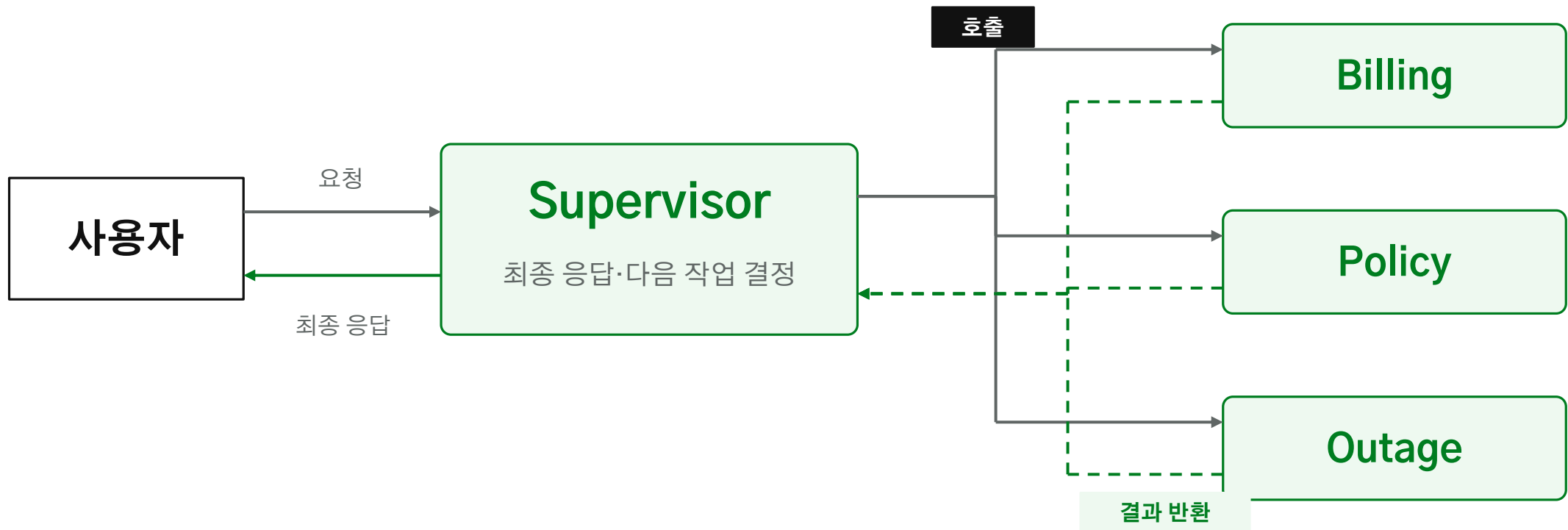
Network는 연결의 모양, 이 강의의 Swarm은 동적 Handoff 규칙에 초점을 둡니다.



한 시점의 active agent와 전환 조건을 명시해야 합니다.

Supervisor | 전문가를 호출하되 최종 제어를 유지합니다

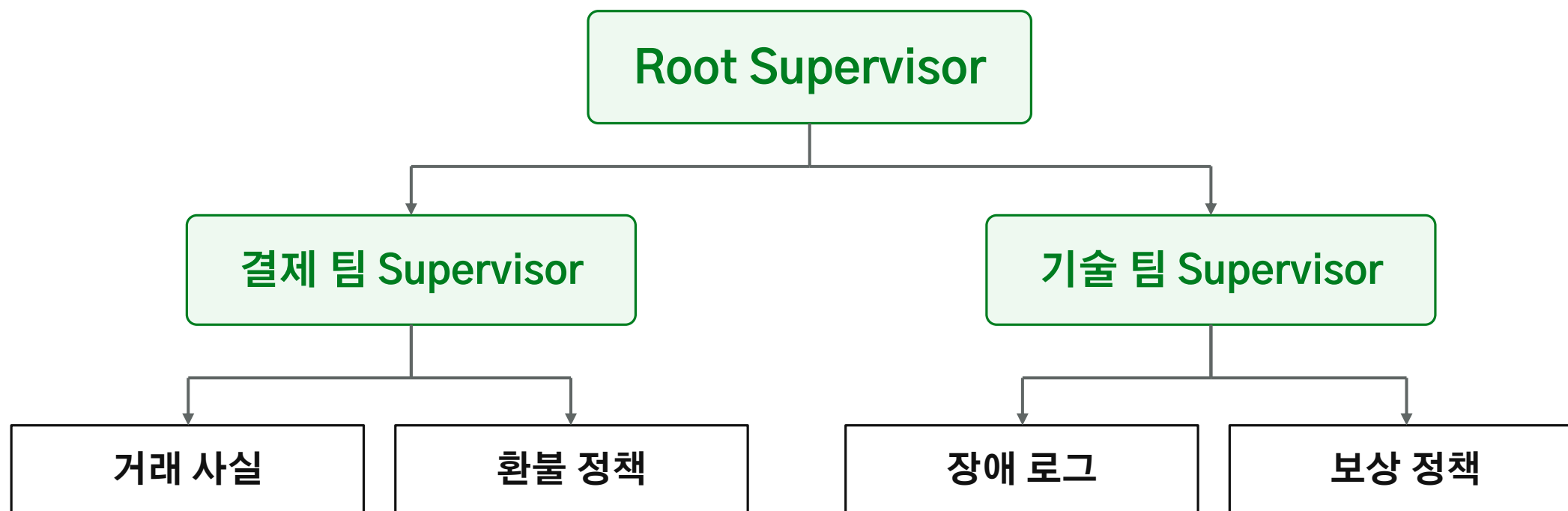
전문가의 결과를 받은 뒤, 다음 작업과 최종 응답을 결정합니다.



전문가는 결과를 반환하고, 대화의 제어권은 Supervisor에 남습니다.

Hierarchical Supervisor | 계층을 추가할 이유가 있어야 합니다

조직도를 모방하기보다, 작업과 정보의 범위가 달라지는지 확인합니다.

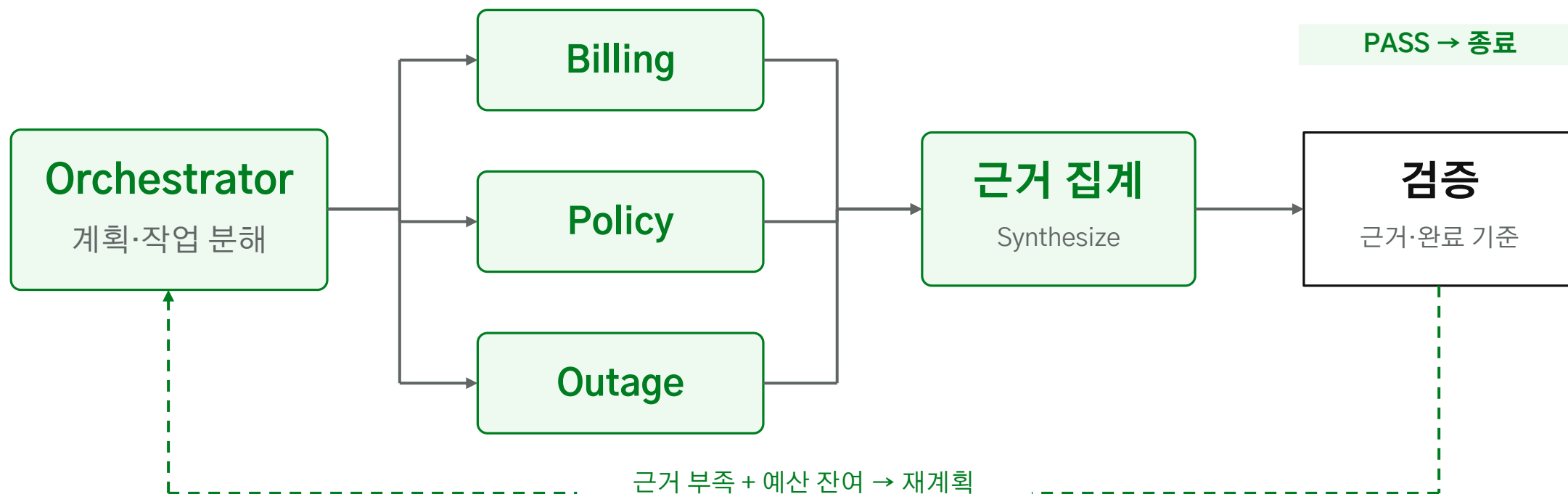


계층 간 전달물: 전체 로그보다 근거·상태·한계

계층이 늘수록 전달 누락·지연·책임 분산도 함께 점검합니다.

Orchestrator-Worker | 작업을 나누고 결과를 다시 합칩니다

요청마다 필요한 작업을 정하고, 독립적인 부분만 병렬로 수행합니다.

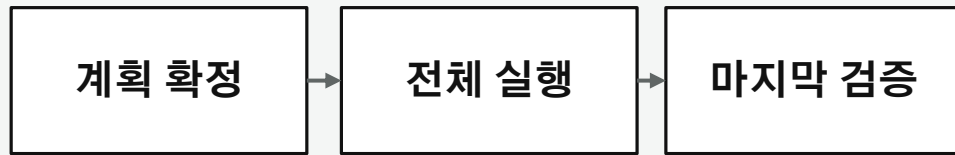


Worker 수를 늘리는 것보다 작업의 독립성과 결과 계약이 중요합니다.

Plan-and-Execute는 반드시 Waterfall이 아닙니다

계획의 존재보다, 실행 중 관찰과 재계획이 가능한지가 중요합니다.

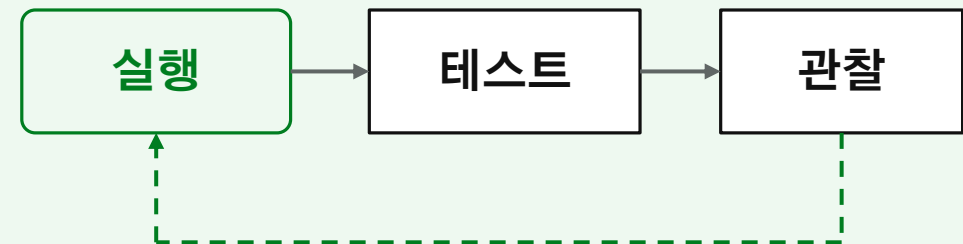
고정 계획·일괄 실행



변화에 대한 피드백이 늦음

관찰·수정·재계획

목표·완료 기준·현재 상태

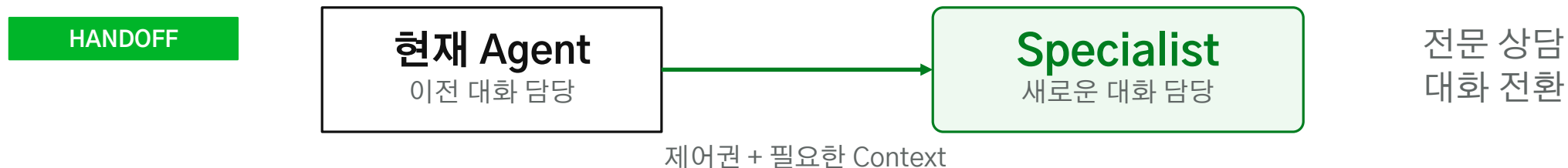


수정 + 계획 갱신

한 Agent도 계획·실행·검증·재계획을 반복할 수 있습니다.

결과를 돌려받을까요, 대화를 넘길까요?

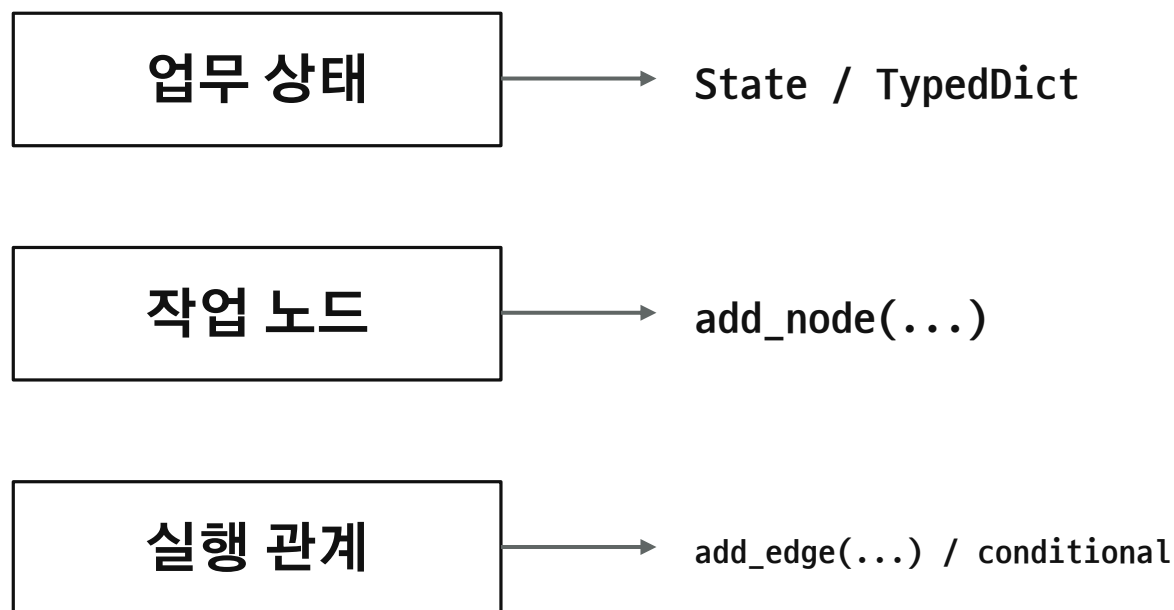
Manager 방식은 결과를 모아 답하고, Handoff 방식은 담당자가 대화를 이어받습니다.



둘 중 하나만 고를 필요는 없습니다. 대화 전환 안에서 조사 Workflow를 쓸 수 있습니다.

LangGraph에서는 구조가 State·Node·Edge로 드러납니다

상태는 무엇을 공유하는가, 노드는 무엇을 하는가, 엣지는 언제 실행되는가를 정합니다.



동적 작업 배분

```
Send("worker", task_input)
```

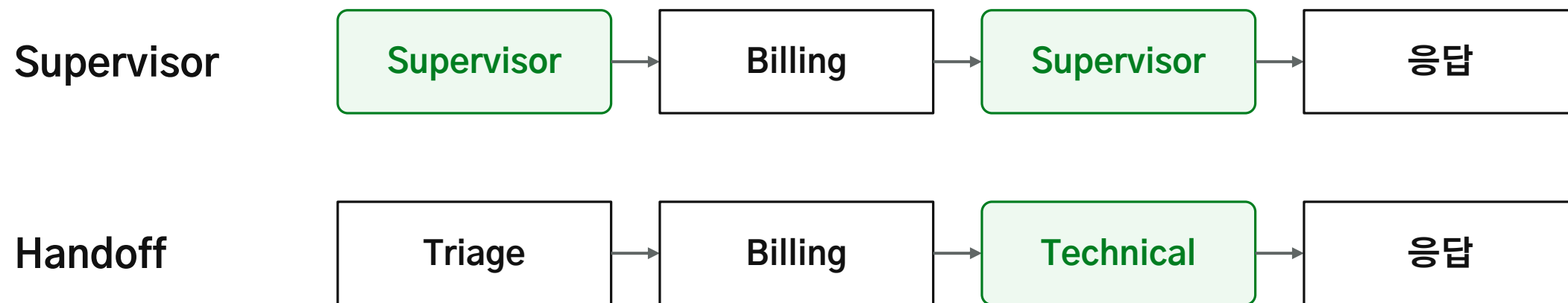
최소 입력 전달
결과 병합 규칙 명시
예산·종료 조건 설정

Send는 동적 작업 배분 수단입니다. 그 자체가 Multi-Agent 아키텍처는 아닙니다.

같은 요청의 실행 경로를 비교합니다

누가 다음 행동을 결정했는지, 결과가 어디로 돌아갔는지 표시합니다.

개념 TRACE · 실측 아님



기록 항목: control owner / 호출 수 / Context 전달 범위 / 최종 응답 주체

두 구조의 실제 Trace와, 제어권이 달라진 지점 1개.

Chapter 3.

Single Agent? Multi Agent?

Multi-Agent를 만들기 전에 Single Agent의 실패를 확인합니다

이미 해결되는 작업을 굳이 나누지는 않습니다. 실패의 원인부터 구분합니다.

관찰한 문제	먼저 점검할 것	분리 검토의 근거
정보가 부족함	검색·조회·추가 질문의 적절성	서로 다른 근거 조사가 필요한가?
도구를 잘못 선택함	도구 설명·입력 계약·권한	도구 범위를 독립시킬 이유가 있는가?
Context가 너무 큼	불필요한 이력·로그·검색 결과	별도 작업으로 분리 가능한가?
단계 의존성이 강함	현재 상태·순서·종료 조건	분리보다 고정 경로가 적합한가?

빈도와 영향도는 기준선 실행 후 기록합니다. 이 표는 실제 오류 통계가 아닙니다.

프롬프트·도구·검색으로 해결할 문제와, 작업 분리가 필요한 문제를 구분합니다.

Multi-Agent의 이득과 비용은 함께 증가할 수 있습니다

작업 분리로 얻는 이득이, 추가 호출과 조정 비용을 정당화해야 합니다.

얻을 수 있는 이득

독립 작업 병렬화

동시에 진행할 수 있는 조사

집중된 Context

작업에 필요한 정보만 제공

전문화·검증 지점

서로 다른 도구와 완료 기준

추가로 지불하는 비용

모델 호출·토큰

Agent 간 지시와 결과도 비용

전달 손실·조정 지연

요약 누락과 반복 호출

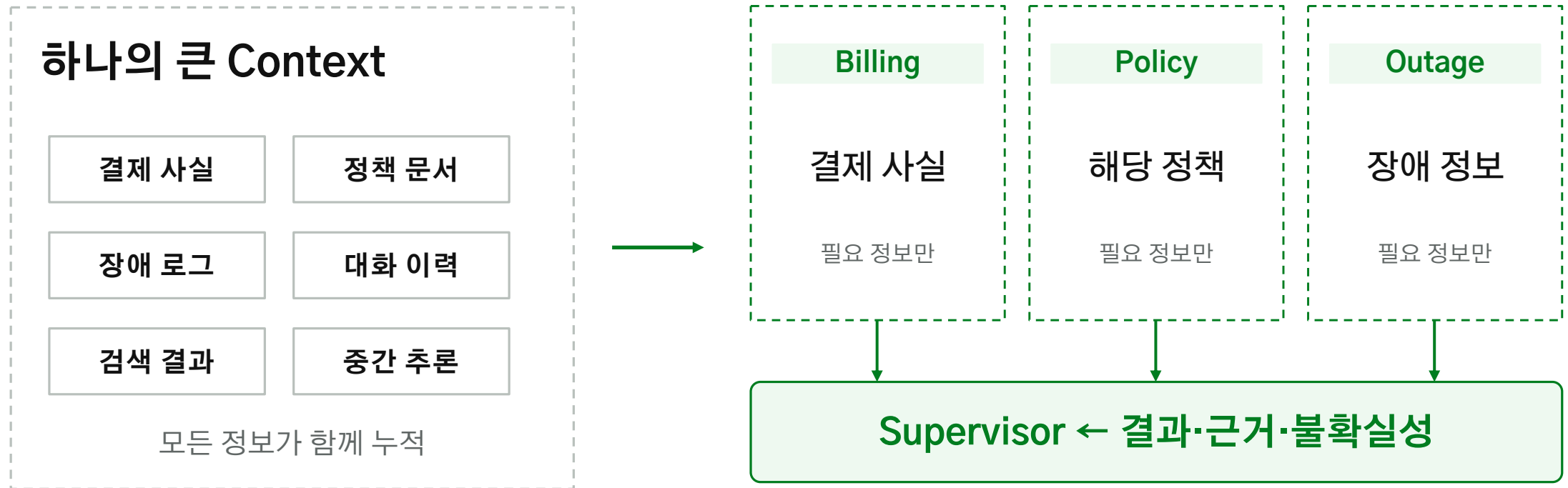
추가 실패 지점

공유 상태·재시도·권한 관리

Agent 수가 아니라, 같은 업무에서 얻은 품질·시간·비용의 변화를 비교합니다.

SubAgent를 나누는 기준은 Persona보다 Context 경계입니다

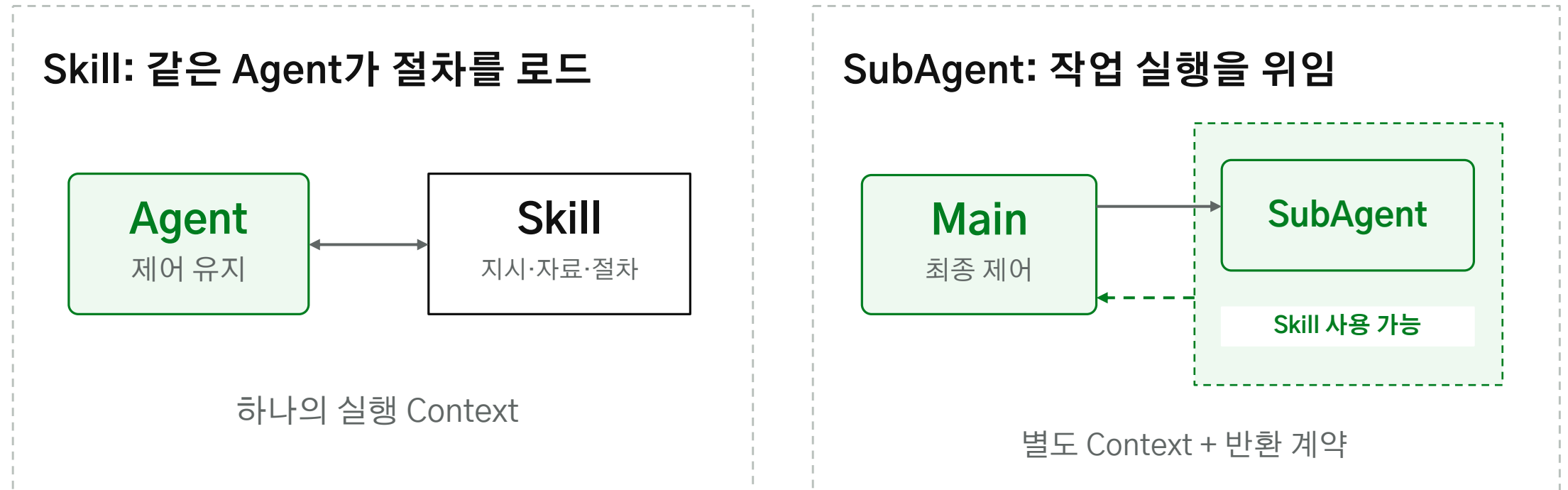
각 작업에 필요한 정보만 주고, 근거와 한계를 포함한 결과를 받습니다.



Context 분리와 보안 격리는 다릅니다. 권한은 별도로 설계합니다.

Skill과 SubAgent는 대체재이면서 조합 가능한 도구입니다

절차를 불러올 것인가, 별도 실행 Context로 위임할 것인가?



복잡도만으로 나누지 말고, Context·도구·권한·병렬 실행의 필요성을 확인합니다.

성능 향상 수치보다 먼저 비교 조건을 읽어야 합니다

아래 두 수치는 같은 비교 기준을 사용하지 않습니다.

Research 평가

+90.2%

비교 대상: Single-Agent Opus 5

Opus 5 리드 + Sonnet 5 SubAgents
특정 내부 research eval의 상대 개선

토큰 사용량

약 15배

비교 대상: 일반 Chat

Multi-Agent 시스템의 토큰 사용 보고
Single Agent 대비 수치가 아님

“90.2%p 향상”도, “Single Agent보다 토큰 15배”도 아닙니다.

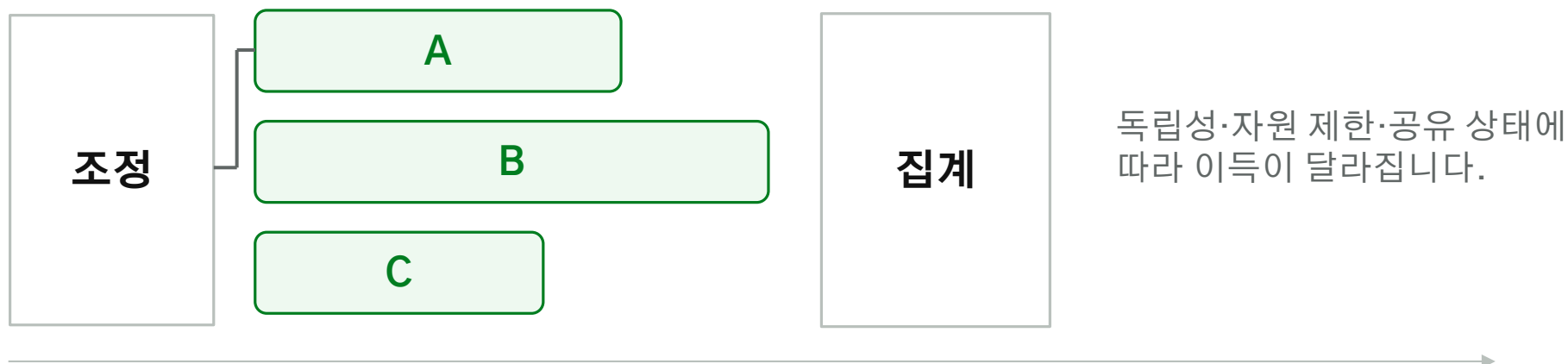
Latency는 Agent 수보다 가장 긴 실행 경로로 봅니다

직렬 작업은 누적되고, 병렬 작업도 조정·집계 시간을 지불합니다.

직렬



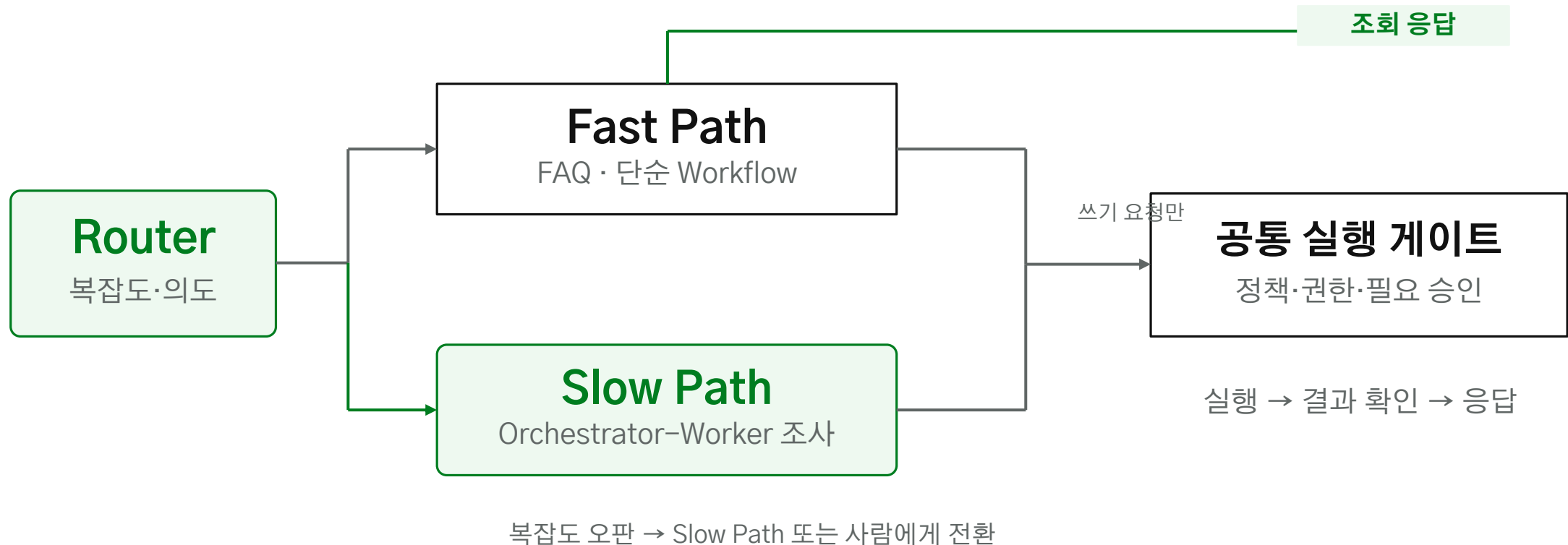
병렬



독립 작업만 병렬화하고, 실제 전체 지연시간으로 확인합니다.

단순 문의와 복합 문의에 같은 비용을 쓸 필요는 없습니다

Fast Path와 Slow Path를 나누되, 쓰기 작업의 안전성 검사는 공통으로 둡니다.



Fast Path는 안전 검사를 생략하는 지름길이 아닙니다.

같은 문제·같은 제약으로 다양한 구조를 비교

성공률만 보지 말고 지연시간·토큰·정책 위반도 함께 기록합니다.

사례 유형	Single	Single + Skills	Supervisor	Handoff	Hybrid
단순 문의	측정	측정	측정	측정	측정
복합 문의	측정	측정	측정	측정	측정
정보 부족	측정	측정	측정	측정	측정
승인 필요	측정	측정	측정	측정	측정

고정 조건

동일 데이터 · 도구 권한 · 모델 조건 · 예산 정책 · 반복 실행

유형별 결과와 선택 근거를 확인합니다

Chapter 4.

무슨무슨 Engineering

새로운 이름은 이전 기술을 없애는 것이 아닙니다

Prompt는 지시, Context는 제공 정보, Harness는 실행 환경을 설계합니다.

Prompt

무엇을 어떻게 지시할까?

Context

매 순간 어떤 정보를 보여줄까?

Harness

어떻게 실행·검증·중단·복구할까?

함께 보는 관점

Graph

작업·상태·의존관계

Loop

반복·재시도·종료

Prompt | 역할보다 작업·근거·완료 기준을 구체화합니다

무엇을 할까, 무엇을 근거로 판단할까, 어떤 형식으로 끝낼까?

역할만 있는 지시

“너는 환불 전문가야.”

역할은 있지만
작업 계약은 부족합니다.

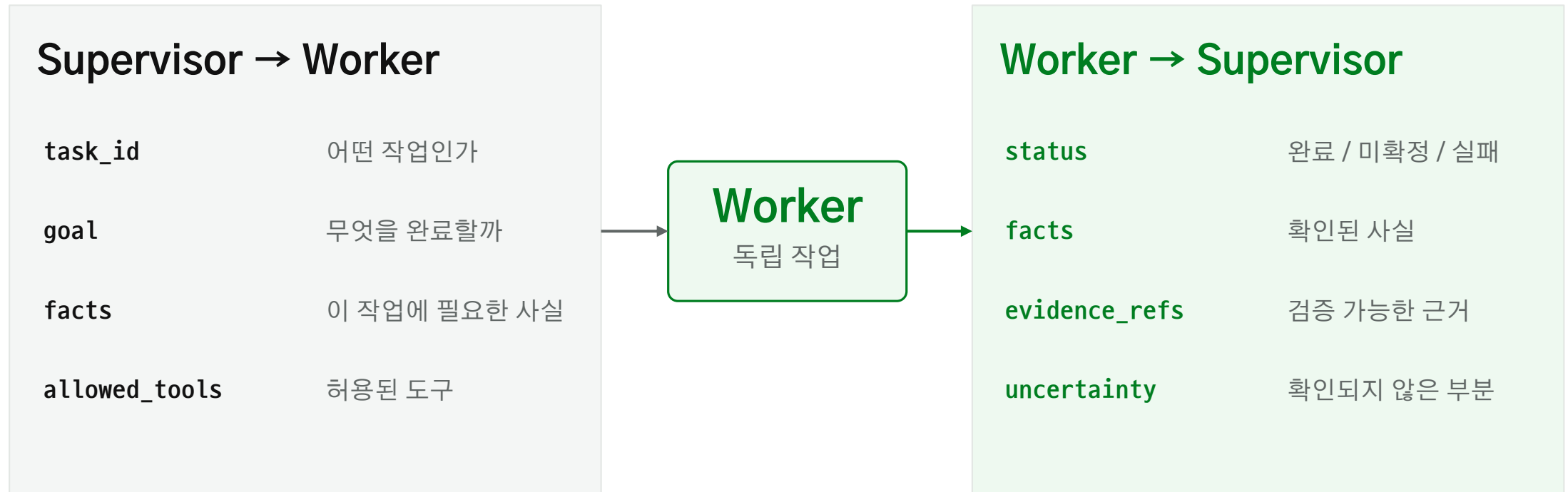
작업 계약이 있는 지시

작업	이중 결제 여부와 보상 가능 근거를 확인
근거	제공된 거래 사실·정책·장애 자료만 사용
출력	확인된 사실 / 근거 / 미확정 항목을 반환
한계	자료가 없으면 확인 불가로 명시
권한	실제 환불은 제안만 하고 실행하지 않음

역할 프롬프트가 무의미한 것이 아니라, 역할 이름만으로 업무가 완성되지 않습니다.

Context | 전달할 정보와 반환할 결과를 설계합니다

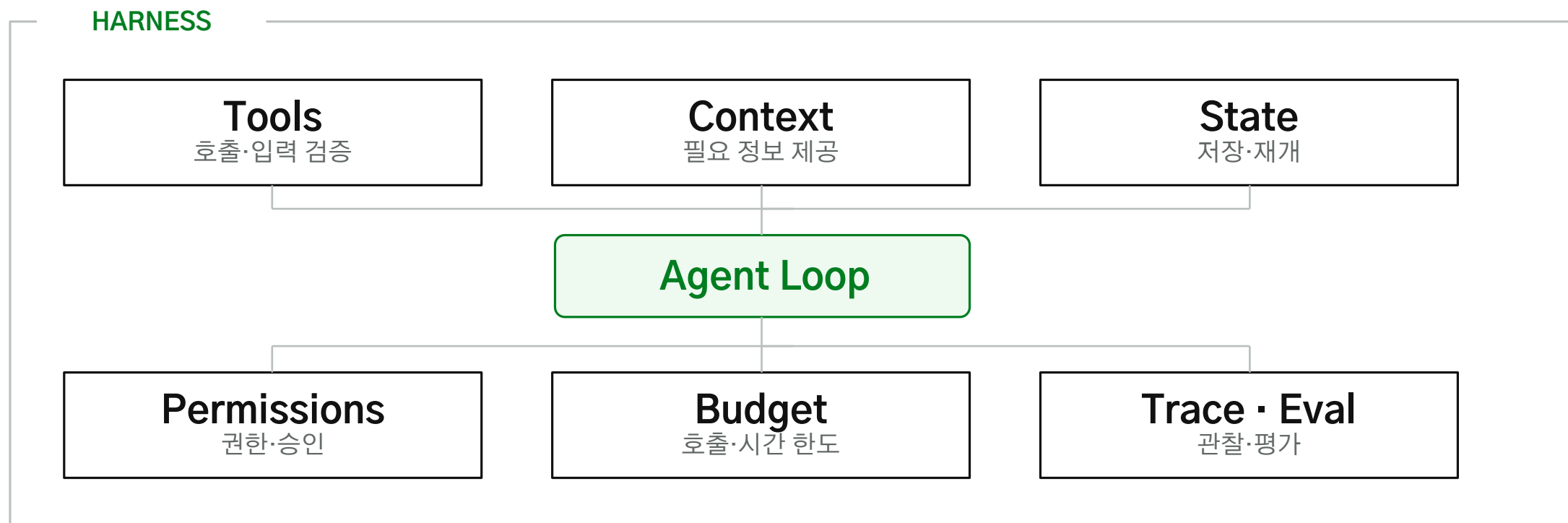
원문 전체를 복제하지 말고, 필요한 입력과 검증 가능한 결과를 전달합니다.



짧은 요약만 받기보다, 사실과 근거와 불확실성을 함께 받습니다.

Harness | 모델이 실제 일을 끝내도록 실행을 관리합니다

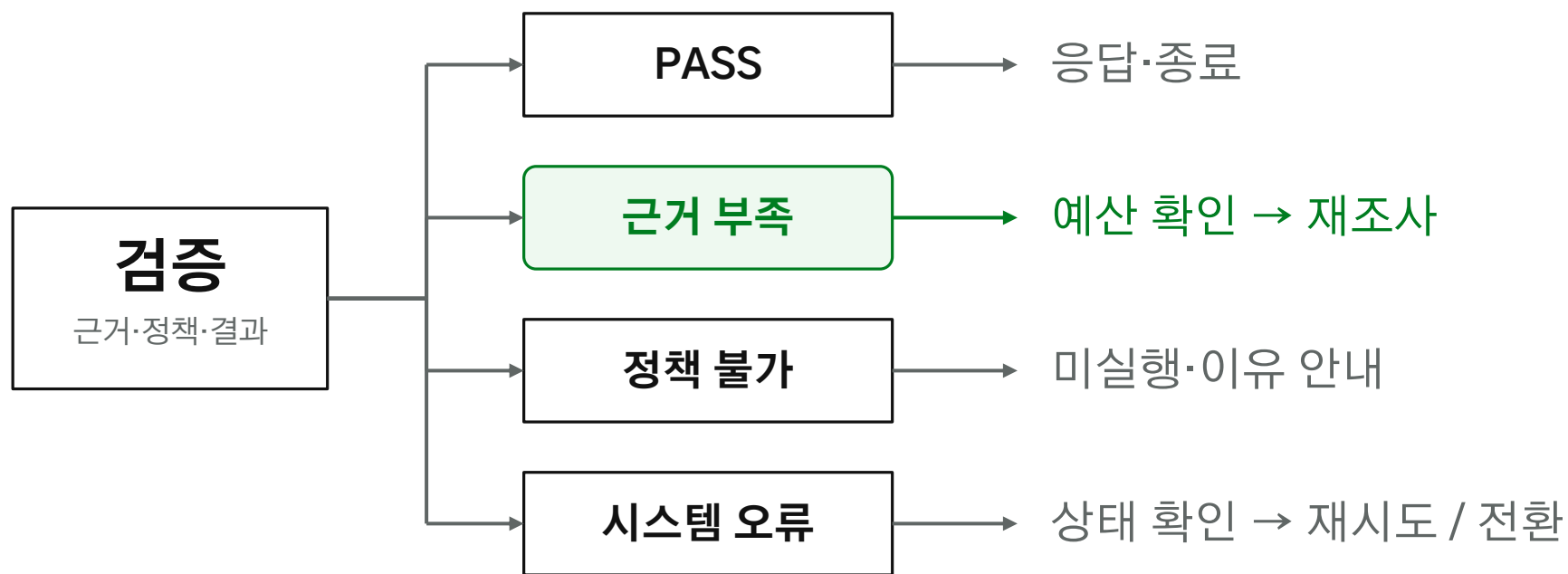
실행·검증·중단·복구는 모델 주변 시스템의 책임입니다.



좋은 프롬프트에 더해, 상태와 검증 기준을 가진 실행 환경이 필요합니다.

Graph와 Loop는 다음 단계와 종료 조건을 명시합니다

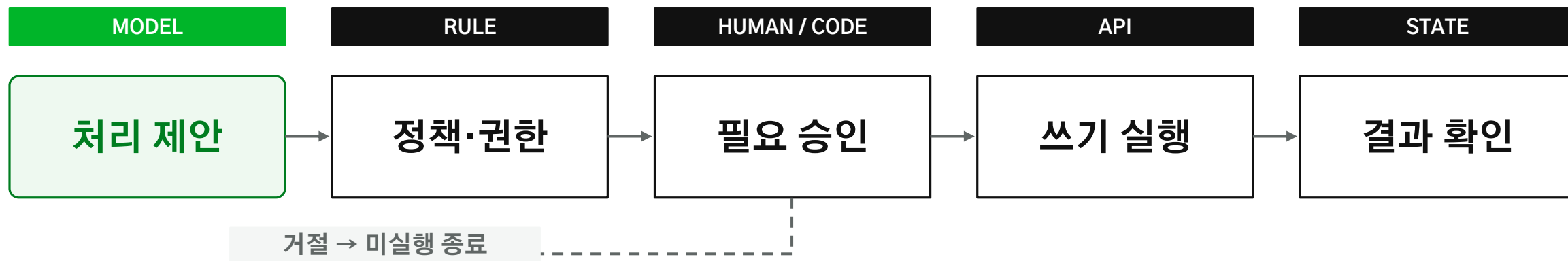
실패했다고 무조건 다시 호출하지 않습니다. 실패 종류에 따라 다음 행동이 달라집니다.



재시도 횟수뿐 아니라, 전체 예산과 미실행·전환·종료 상태를 정합니다.

승인은 실행 전에, 재시도는 중복 실행 없이 설계합니다

처리 제안 → 정책·권한 검증 → 필요한 승인 → 쓰기 실행 → 결과 확인



실행 전후: Checkpoint + 처리 상태

중복 방지: 요청 키 + 외부 결과 조회

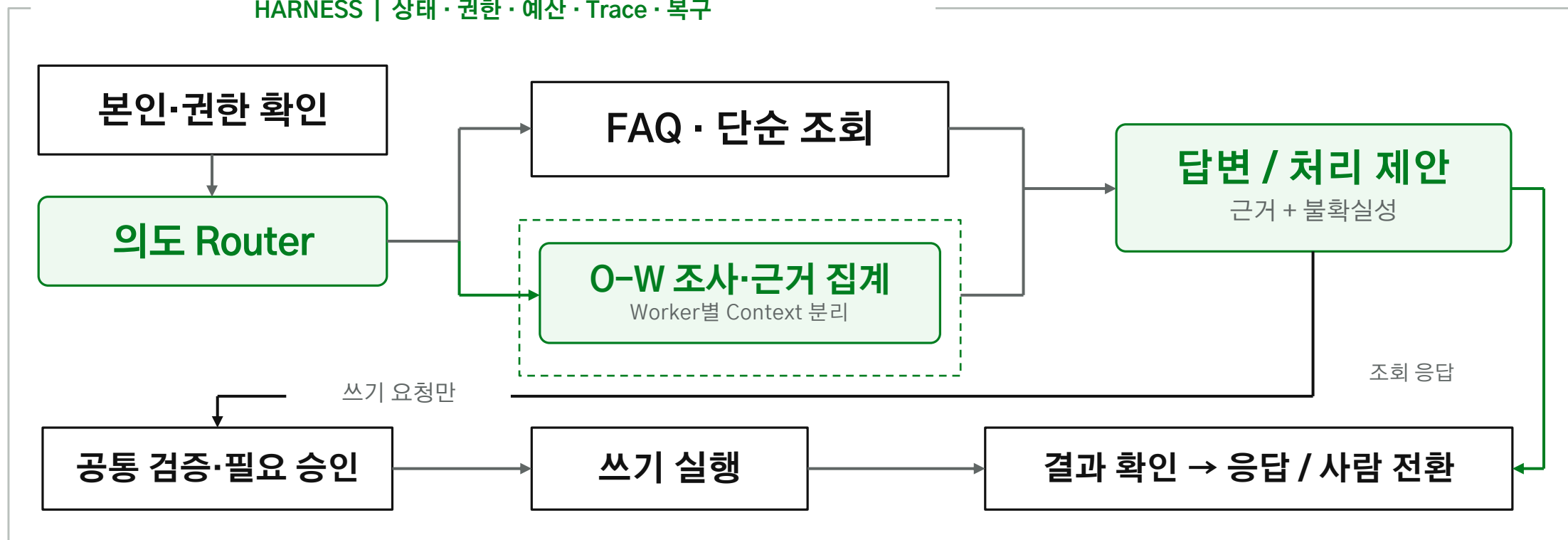
idempotency_key는 외부 시스템이 지원·강제해야 합니다.

Checkpoint로 재개해도, 외부 API의 exactly-once 실행이 자동 보장되지는 않습니다.

완성형 시스템 | Workflow 안에 필요한 Agent만 배치합니다

Fast/Slow 분석 경로 + Context 분리 + 공통 실행 게이트 + 검증·복구

HARNESS | 상태 · 권한 · 예산 · Trace · 복구



처음의 업무 목표를 끝까지 추적하고, 실행과 미실행의 결과까지 설명합니다.

마지막 질문 | 왜 이 구조를 선택했고, 어떤 장-단점이 있나요?

업무 목표

어떤 상태를 완료로 볼까?

작업 Graph

순차·병렬·분기·루프는?

Agent·SubAgent의 필요성

코드나 Skill로 충분하지 않은 이유는?

Context·권한

무엇을 보고 무엇을 실행할 수 있을까?

예산·종료

언제 재시도하고 언제 전환할까?

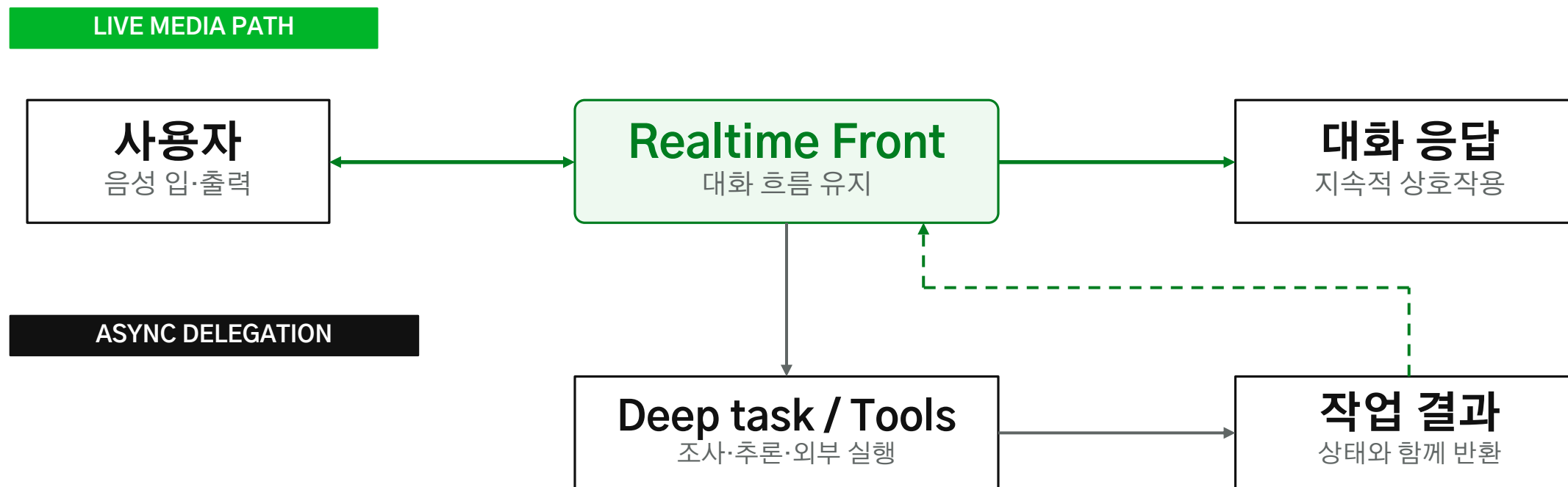
평가 근거

어떤 결과로 선택을 정당화할까?

Agent 수가 아니라, 설계 이유가 결과물입니다.

부가정보: 실시간 대화와 깊은 작업을 분리할 수 있습니다

대화의 미디어 경로를 유지하면서, 깊은 조사와 도구 실행은 별도 비동기 경로로 보냅니다.



Fast/Slow는 “어떤 경로인가”, 비동기는 “기다리는 동안 무엇을 할까”의 문제입니다.